Helping Eyes

A camera reading assistant for visually impaired people: point, ask, and hear only what you need



Neeraj P Rao (1RVU23CSE303) · Parveen (1RVU23CSE330) · Misha (1RVU23CSE269)

Guide: Prof. Manjul Krishna Gupta · RV University

CS3235 · Working with Large Language Models · November 2026

Apple Vision OCR · OpenCV · Gemini / Qwen · Kokoro voice

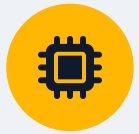
Printed text is still a barrier

Existing reading aids fall short in five ways



Cost

Most commercial reading aids are expensive.



Dedicated hardware

They need special devices, not the phone or laptop people already own.



Low light

Real-time reading in poor light is hard for traditional OCR.



Accuracy

Blur, busy backgrounds and varied fonts break simple OCR pipelines.



Information overload

Reading a whole label aloud means listening to everything to find one detail.

Ask, don't listen to everything

Capture once, then ask for exactly the part you need

The camera finds the text by itself, coaches you until it is in view, and captures it. Then you speak or type a request and hear a short answer taken only from that text.

Pages · books · medicine packs
food labels · signs · screens

“Read only the dosage”
Reads just the requested part

“When does it expire?”
Date compared with today in code, then spoken

“Is this safe for children?”
Answered from the printed text, or “the text doesn't say”

“Read everything”
Full text exactly as recognised, no AI model needed

“Look it up online”
Web search only after you agree

Objectives

01



Detect text automatically

From a live camera feed, on any object

02



Read accurately and fast

With Apple Vision, built into macOS

03



Ask for what you need

Instead of listening to the whole label

04



Never invent details

Answer only from the captured text

05



Read books hands-free

Page by page, no buttons between pages

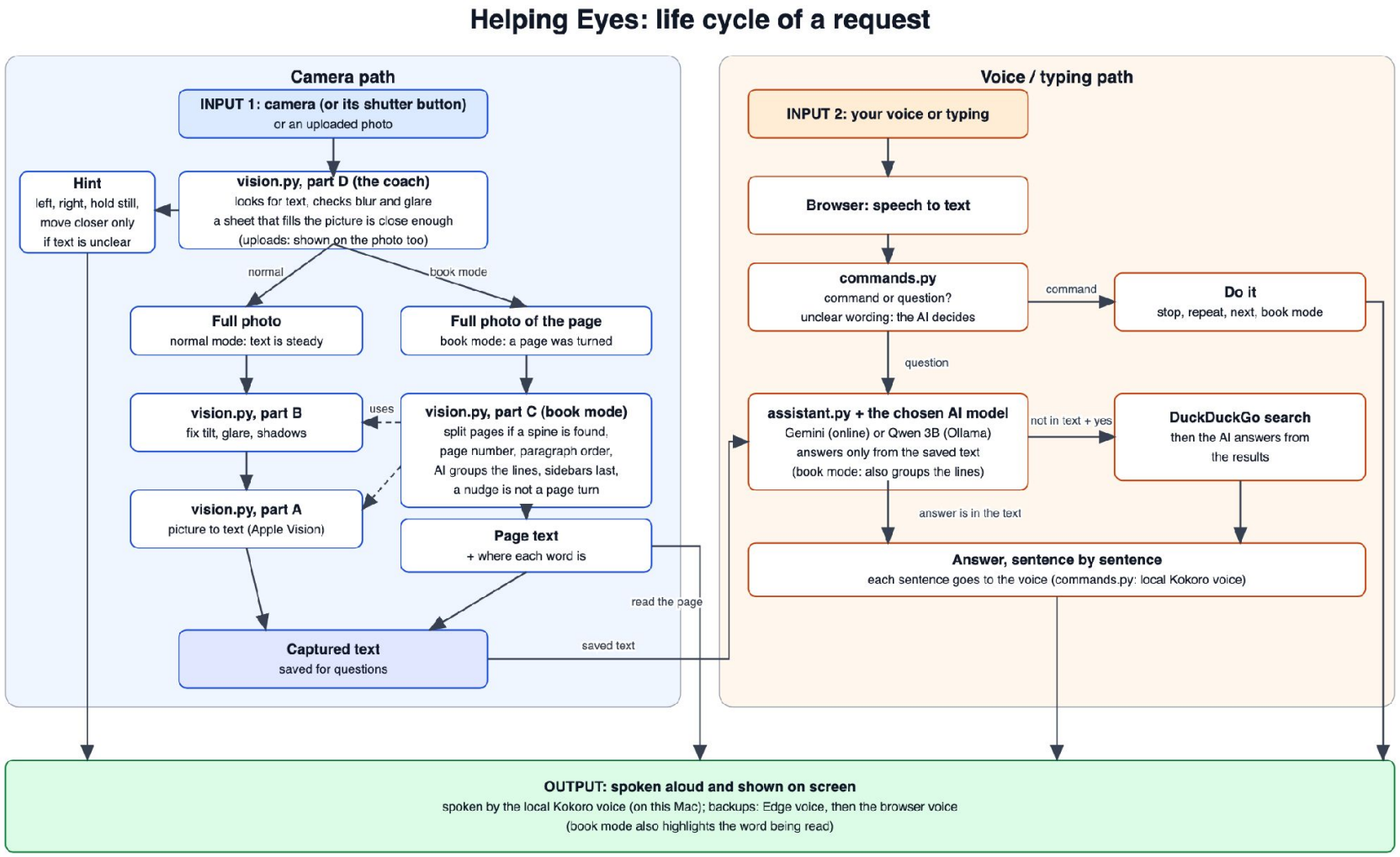
06



Ordinary hardware

A Mac, any browser, a free model key: no separate GPU, nothing paid

Architecture / Life cycle of one request



Two AI models, web search and a local voice

The browser handles the camera, microphone and speaker; the Mac does the reading and answering

Gemini Flash-Lite (online AI)

The default answer model, used through Google's free API key. Only the captured text is sent, never the photo.

Qwen 2.5 3B (offline AI)

Runs on the Mac through Ollama. No key and no internet needed, and no text leaves the Mac. Faster, but less accurate.

DuckDuckGo (web search)

Used only when the label can't answer and the user says yes. Only the question goes out; the answer starts with “According to the web”.

Kokoro-82M (voice)

Speaks every answer, made on the Mac with no internet. If it fails, the browser's own voice takes over.

Capture: coached into view, then read

The camera finds the text by itself, talks you into position, takes the photo and reads it



Spoken hints

“Move left”, “Hold still”, “Glare on the right, tilt the page”. Spoken until the text is in view.

Capture

Steady and in view: short hold-still countdown, then automatic capture. The shutter button or key C captures at once.

Behind the scenes: OpenCV checks blur, glare and light, then straightens and cleans up the photo so Apple Vision can read it.

90.5 % of key lines read correctly on 164 real photos

0.47 s to capture a label (9 in 10 photos this fast or faster)

Grounded answers

assistant.py keeps the model to the captured text

Only the text leaves the Mac

The captured text is sent to the model, never the picture. Answers stream sentence by sentence.

“The text doesn't say”

When a detail is not printed, the model says so instead of guessing.

Expiry dates checked in code

Dates are compared with today in code; the model only words the verdict. Dates printed away from their label are checked too.

Web lookup only with consent

Offered when the text can't answer; starts “According to the web”. Never offered for expiry, batch or price.

Answers from…

Online AI (Gemini Flash-Lite)
Default. Free key; any OpenAI-compatible API works by changing three settings.

Offline AI on this computer (Qwen 2.5 3B)
Runs through Ollama on the Mac. No key, no rate limit, no text leaves the Mac. run.py starts and warms it.

Chosen per request in the page's menu and remembered by the browser.

Voice in, voice out

Hands-free listening and a natural voice that runs on the Mac

Speech out: Kokoro first, the browser voice if it fails

Kokoro-82M
On the Mac, no internet · about 1.3 s a sentence

→

Browser voice
Built into the browser · used only if Kokoro fails

Listens only while the app is quiet Browser speech recognition, hands-free; the microphone closes while the app talks, so it never answers itself.

No stalls Next sentences are made while one plays, so there is no pause between them.

Numbers read clearly Serial and batch numbers above 9999 are read digit by digit.

Book mode: read a book without touching anything

Open the book, listen, turn the page

Watch

Frame differencing until the view is still for 0.8 s

Find the book

Border and spine; split the two-page spread

Read

Apple Vision lines and word boxes

Page facts

Header and footer: page number, running title

Order

Recursive XY-cut: columns, headings, paragraphs

Paragraphs

Model groups lines and fixes misreads

Speak

Announce the page, read, highlight each word

On screen: the book's border in red, the spine in yellow, page numbers and title in purple, paragraphs in green, numbered in reading order. The line being spoken is highlighted and the word being spoken is outlined.
Book nudged: reading carries on from the exact word it reached.

Built to be used without seeing the screen

The page itself follows WCAG; axe-core reports no issues, including the AAA contrast rule

7:1 contrast

All text, in light and dark themes; overlay labels on a dark plate

Screen-reader mode

Everything the app would say goes to the screen reader or braille display

Single-key shortcuts

C capture · N new · B book · A all · R repeat · S stop · L listen; can be turned off

Large targets

48 px buttons on phones, 3 px focus ring, on/off shown by a check mark, not colour alone

Problems said aloud

Camera blocked, no microphone, server unreachable, connection lost and back

Any screen

Fits a laptop with no scrolling; phones scroll; follows reduced motion and high contrast

Evaluated on real photos, not a benchmark

Tested on 28 and 29 October 2026 on an Apple M3 Pro MacBook

164

real phone photos

148 medicine packs and 16 other items, in good and dim light, flat, tilted and curved

708

questions to each model

on 46 of the photos, asked the same way the user asks them

22

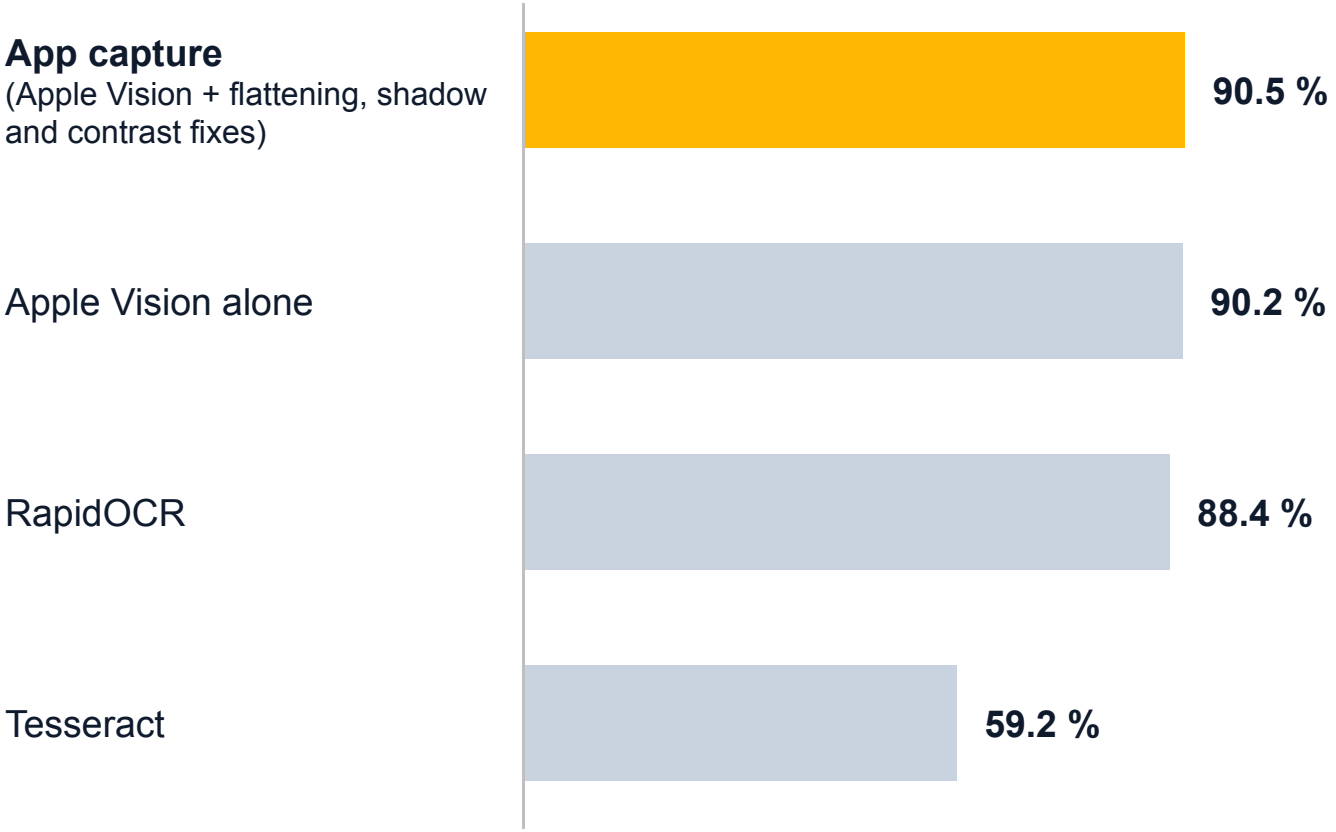
expiry label texts

written like real pack dates; half already expired

How answers were checked. For every photo, the developer wrote down by hand what the label really says, before looking at anything the app produced. Every answer was compared with that. Short facts such as dates, batch numbers and prices were marked right or wrong by fixed rules; longer answers were checked on a random sample without knowing which model wrote them.

OCR: Apple Vision reads 90 % of key lines

Key-line accuracy on 164 real photos



The top three are too close to call; Tesseract is far behind.

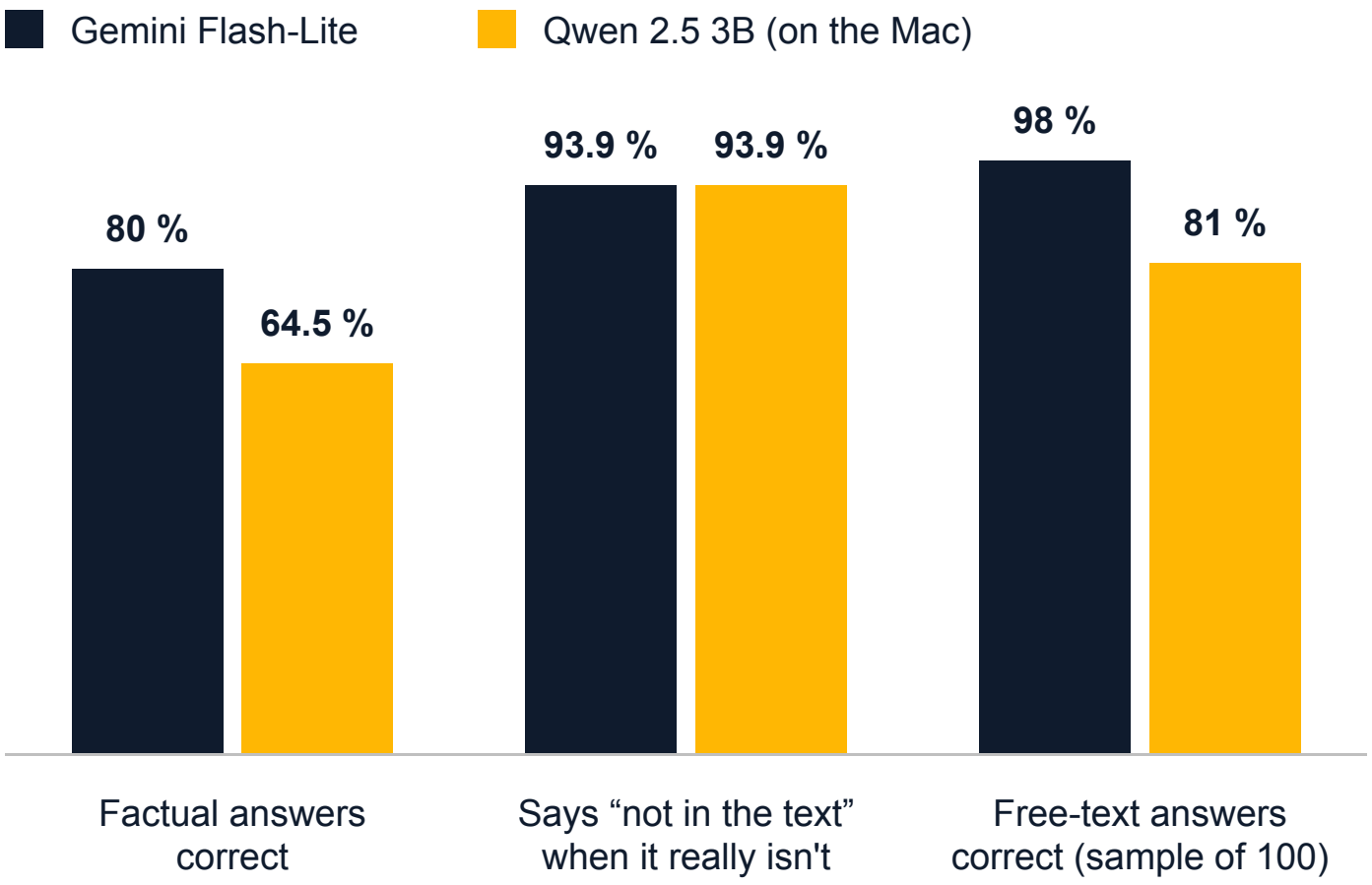
By kind of item (app capture)

Documents and screens	**100 %**
Medicine cartons	**97 %**
Book pages	**96 %**
Food labels	**86 %**
Foil tablet strips	**81 %**
Curved items	**81 %**
Dim light (2 photos)	**76 %**

Target: character error rate ≤ 0.05. Measured 0.095: not met.

Answers: Gemini is more accurate, Qwen is faster

The same 708 questions on 46 photos, through the app's own code



+15.5 pts

Gemini over Qwen on the same 355 factual questions (95 % interval 11 to 21)

0.45 s

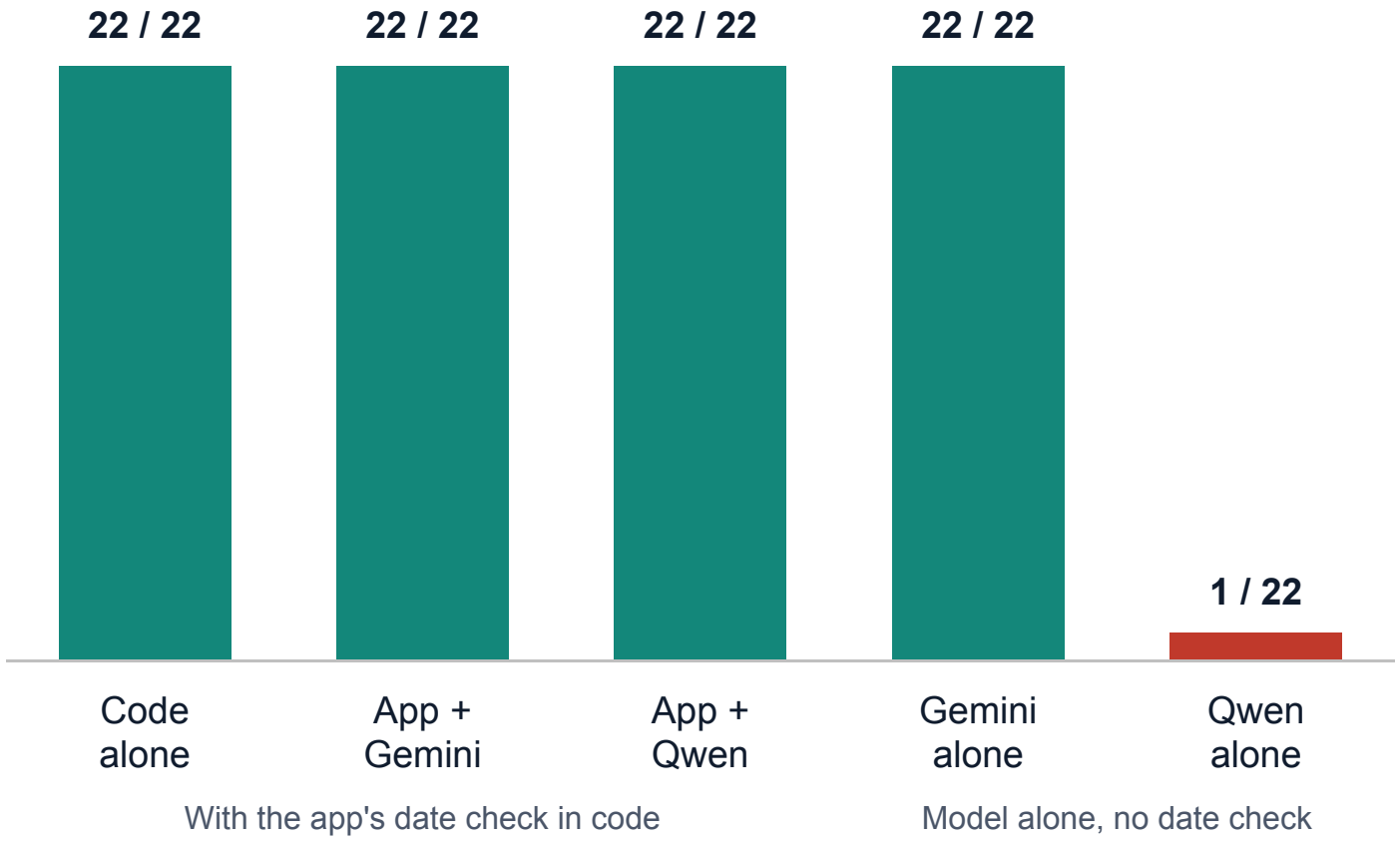
Qwen's typical time to the first sentence (Gemini 3.1 s on a busy free tier)

≤ 2 %

answers with a number that is not in the text, both models

Expiry: checking dates in code is what makes it safe

“Has it expired?” on 22 label texts, half of them expired



Qwen alone: 1 / 22

Without the code check, all 22 of its answers began “The text doesn't say…” although every label prints its date.

Dates printed away from their label

On 34 real packs whose dates sit in a separate column, the right expiry date went from 0 to 21 (Qwen) and 22 (Gemini) once every printed date was checked.

Speed

Each test was run 20 times. Typical time: the middle run. Slow case: only 1 run in 10 was slower.

Operation	Typical time	Slow case (1 in 10)
Capture of a label (1920 px photo to text)	0.29 s	0.47 s
Question to start of spoken answer, Qwen	1.37 s	1.89 s
Question to start of spoken answer, Gemini (busy free tier)	9.8 s	103 s
Live frame round trip (960 px)	0.11 s	0.14 s
Kokoro voice for one sentence	1.30 s	1.33 s
Book page, Qwen groups the paragraphs	5.8 s	6.1 s
Book page, Gemini groups the paragraphs	3.8 s	12.5 s
Live checks per second (server alone: 9.3)	4.8 / s	–

Known limitations

Reported as found

Mac only

Apple Vision exists only on macOS; the shared link works while the Mac is awake and running.

Web answers

When results were about another company, the model reported them as facts about the item.

Two-column labels

A value printed a row away from its label can be paired with the wrong label.

Phones

iPhone Safari cannot listen hands-free; the first sentence may wait for a tap.

Medicine details should still be confirmed with a pharmacist or doctor: a small model can still misread or mix up details.

Next steps

- Make it accessible for non Mac devices
- More accurate OCR readings
- Correct semantic meaning (tables)
- Add image model

Try it

`python run.py`
checks the setup, starts the server, opens Chrome

`python run.py --share`
public https link for a phone, kept private by an access code

Thank you

github.com/neerajprao/Helping_Eyes